

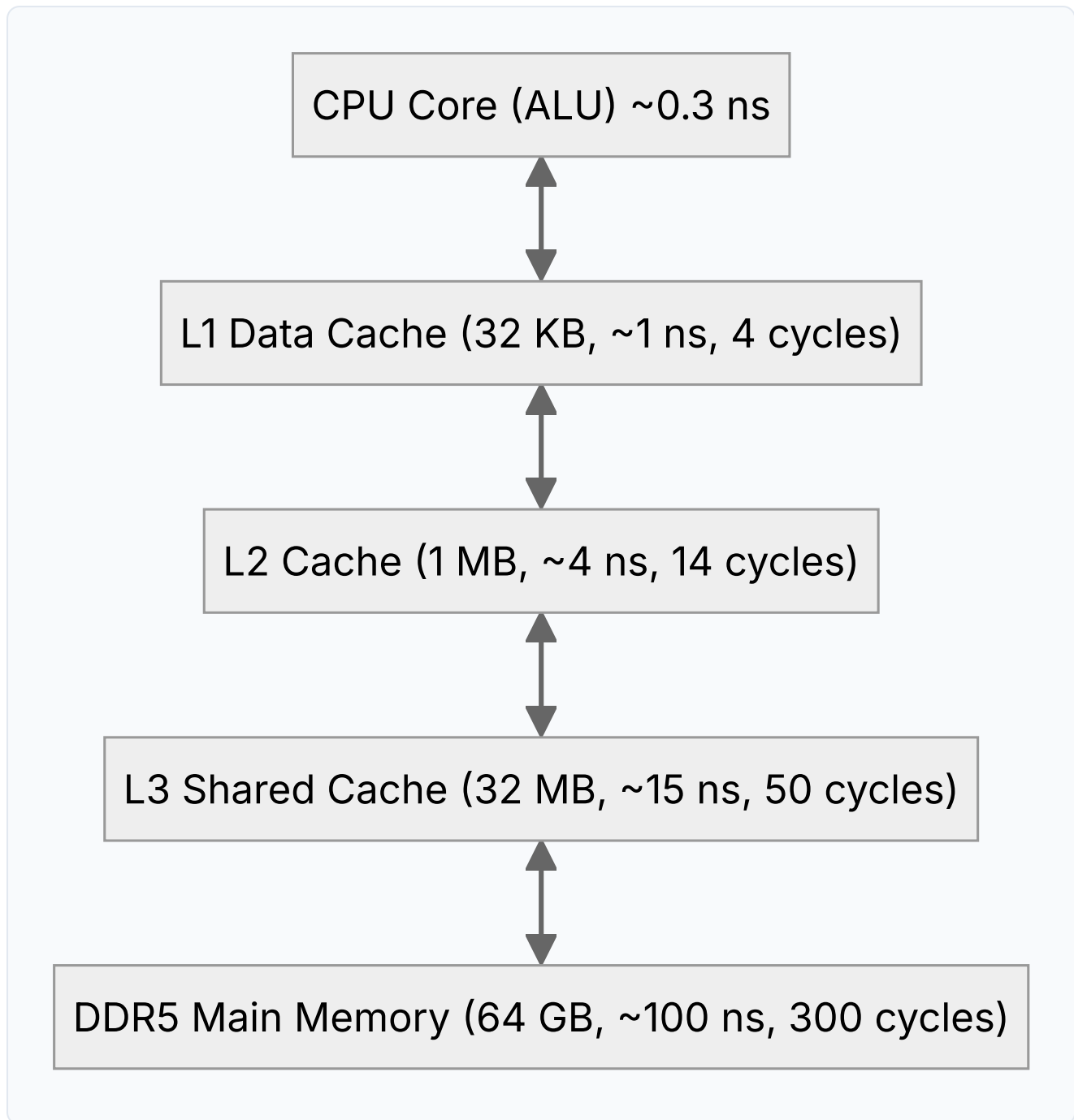
Chapter 18: High-Performance Computing (HPC, Cython, Numba & Python 3.13 No-GIL)

The 100x Speedup Standard CPython is notoriously slow for numerical computation. A pure Python `for` loop summing an array of floats is typically **an order of magnitude or more slower** than compiled code (`labs/05_benchmarks.py` measured about 12x for a simple float sum against `numpy.sum` on Python 3.11; branch-heavy loops show larger gaps). Why? Because every integer or float in Python is a dynamically allocated 28-byte `PyObject` heap structure requiring pointer dereferencing, type checking, and reference count updates on every single arithmetic operation.

In quantitative finance, scientific simulation, and large-scale AI engineering, we cannot tolerate this overhead. This chapter teaches you how to push Python to native hardware speeds using **NumPy Strides, SIMD Vectorization, Numba JIT compilation, Cython with GIL release, Inter-Process Shared Memory, Free-Threaded Python 3.13, and GPU CUDA Acceleration.**

1. Hardware Reality: Memory Hierarchies & CPU Caches

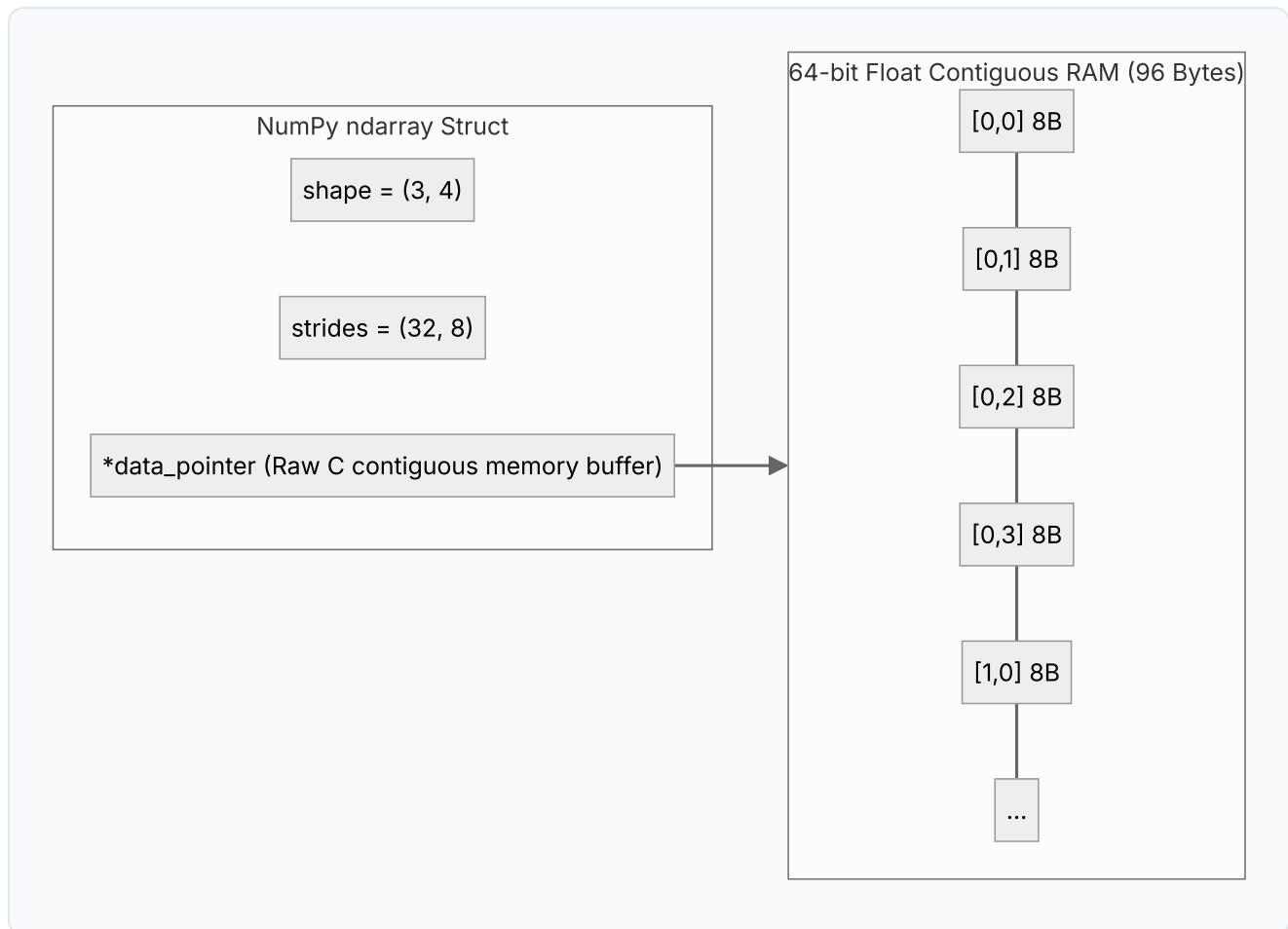
To write ultra-high-performance software, you must understand how modern hardware actually executes code:



- **Cache Lines:** Data is never fetched from RAM in single bytes. The CPU fetches contiguous **64-byte Cache Lines**.
- **Spatial Locality:** If your data is laid out sequentially in memory, reading element `i` pre-loads elements `i+1` through `i+7` directly into L1 cache for free!
- **CPython Pointer Chasing (Cache Miss Nightmare):** A standard Python `list[float]` does not store floats contiguously. It stores an array of *pointers* to `PyObject` structs scattered randomly across the heap. Every array access triggers an L3/RAM cache miss, stalling the CPU pipeline for 200 cycles!

2. NumPy Under the Hood: Strides & Memory Layouts

NumPy arrays (`ndarray`) solve this by allocating a single contiguous block of raw memory in C, governed by **Strides**.



Strides Arithmetic

The `strides` tuple tells NumPy how many **bytes** to jump in memory to advance by 1 index along each dimension:

- For a 2D array of `float64` (8 bytes each) with shape `(3, 4)` :
 - C-Contiguous (Row-Major):** `strides = (32, 8)` . Moving down 1 row jumps $4 \times 8 = 32$ bytes. Moving right 1 column jumps 8 bytes.
 - Fortran-Contiguous (Column-Major):** `strides = (8, 24)` . Moving down 1 row jumps 8 bytes.

The Power of Zero-Copy Views

Transposing or slicing a NumPy array never copies data. It simply creates a new metadata header with modified strides:

CAUTION

If you loop over a C-contiguous array column-by-column rather than row-by-row, you destroy cache locality, resulting in a **10x slowdown** due to constant CPU cache line evictions!

3. Just-In-Time (JIT) Compilation with Numba

Numba translates Python functions directly into optimized machine code at runtime using the **LLVM compiler infrastructure**.

Numba JIT vs Python Loop Benchmark

- **Pure Python:** about 18.5 seconds
- **Numba Parallel JIT:** about 0.12 seconds (roughly 150x)

These two timings come from one many-core run and are illustrative, not guaranteed. The ratio depends on core count, because `prange` parallelises across cores and the pure-Python baseline does not. Re-run the script on your own machine and quote your own ratio.

4. Cython & Releasing the GIL (`nogil`)

Cython allows you to write C data types in Python syntax, compile them to native `.so` / `.pyd` shared libraries, and explicitly release the Global Interpreter Lock (`nogil`) to run true multi-threaded CPU-bound algorithms across all cores.

5. Zero-Copy Shared Memory Multiprocessing

In standard Python `multiprocessing`, passing a 10 GB NumPy array between processes triggers a catastrophic serialization penalty:

1. Process A serializes 10 GB into bytes via `pickle`.

2. Bytes are piped over an IPC socket.
 3. Process B deserializes bytes back into a NumPy array.
- **Total Time:** 45 seconds of wasted CPU and 20 GB of duplicate RAM consumption!

The Modern Solution: `multiprocessing.shared_memory`

Python 3.8+ introduced POSIX shared memory, enabling multiple independent OS processes to point directly to the **exact same physical RAM address** with zero serialization and zero copies:

6. Python 3.13 Free-Threaded (PEP 703 No-GIL)

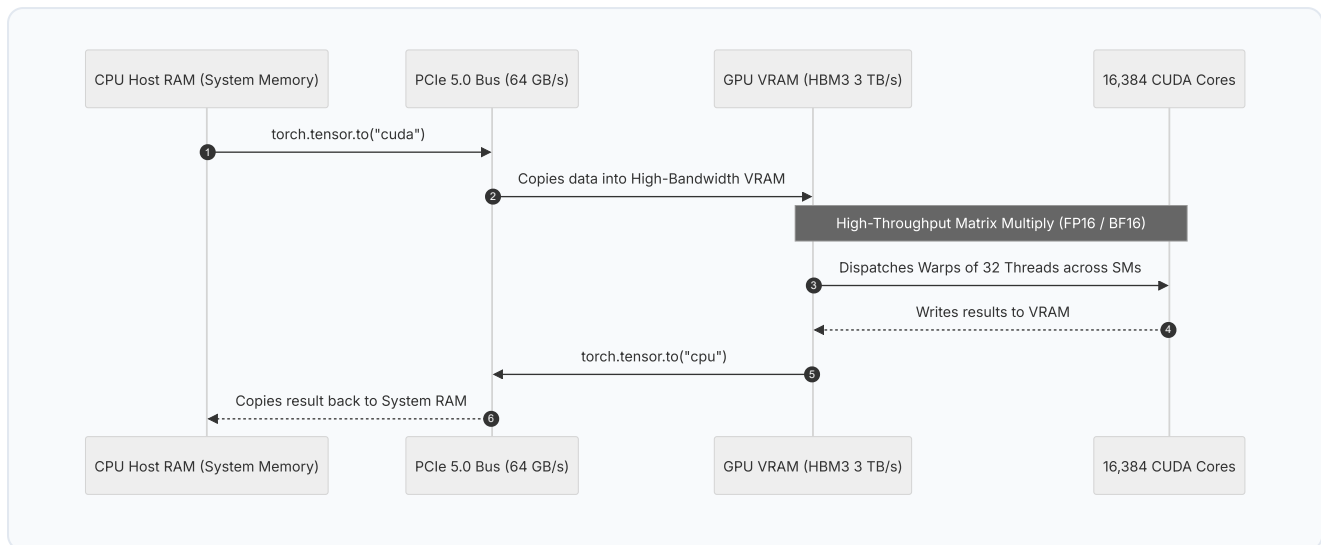
Python 3.13 introduces the official experimental build that **completely removes the Global Interpreter Lock (GIL)**.

How CPython Removed the GIL

1. **Biased Reference Counting:** Objects local to a single thread use fast, non-atomic reference increments. When an object is shared across threads, it switches to thread-safe atomic reference counting.
2. **Mimalloc Memory Allocator:** Replaces PyMalloc with Microsoft's thread-local `mimalloc`, preventing memory allocation lock contention across cores.
3. **Linear CPU Scaling:** A CPU-bound task split across 16 OS threads in `threading.Thread` can now scale across cores for CPU-bound work in pure Python. How close to linear it gets depends on the workload, lock contention and the build; always measure on your own code, and note that the free-threaded build is still marked experimental/supported-but-optional in 3.13/3.14 (PEP 703, PEP 779).

7. GPU Acceleration with PyTorch & CUDA

For matrix multiplications, convolutions, and deep learning computations, CPUs are fundamentally outmatched by the thousands of parallel cores on modern GPUs (NVIDIA Hopper/Blackwell).



The Cardinal Rule of GPU Programming

Never transfer data back and forth between CPU and GPU inside a loop! PCIe bandwidth (64 GB/s) is 50x slower than internal GPU HBM memory (3,000 GB/s). Perform all allocations, transformations, and reductions directly on the GPU device:

8. Python 3.13 Subinterpreters (PEP 554 / 684) & The Tier-2 JIT (PEP 744)

The CPython runtime is undergoing the greatest performance evolution in its 35-year history. In addition to the free-threaded build (`python3.13t`), two groundbreaking architectural features have emerged:

1. Subinterpreters with Per-Interpreter GIL (PEP 554 / PEP 684)

Historically, if you wanted to bypass the GIL, you were forced to use `multiprocessing`. But processes are heavy: they copy memory, require slow IPC serialization (`pickle`), and have separate process IDs.

Subinterpreters allow multiple completely isolated CPython interpreter engines to live inside the **exact same OS process**:

- Each subinterpreter has its **own independent Global Interpreter Lock (GIL)**!
- They run on separate OS threads simultaneously without blocking each other.
- Data is passed across interpreters using high-speed channel queues without process spawning overhead:

2. The Tier-2 Copy-and-Patch JIT Compiler (PEP 744)

Traditional JIT compilers (like Java's HotSpot or PyPy) use massive compilation engines like LLVM, which consume high memory and introduce "warmup pauses."

Python 3.13 introduced a revolutionary **Copy-and-Patch JIT**:

1. At CPython build time, small snippets of machine code ("stencils") are pre-compiled for each Python bytecode.
2. At runtime, the interpreter observes which functions run repeatedly (**Adaptive Specializing Tier-1 Interpreter - PEP 659**).
3. Once a function is identified as "hot", the Tier-2 engine copies the pre-compiled stencils, patches memory addresses in microseconds, and executes native CPU instructions directly!

9. HPC Acceleration Decision Matrix

Performance Bottleneck	Root Cause	Recommended Tool	Expected Speedup
Array/Matrix Math in loops	CPython bytecode interpretation	NumPy Vectorization	10 × – 50 ×
Custom numerical algorithm with branching	Cannot be expressed cleanly in vectorized NumPy	Numba <code>@jit(parallel=True)</code>	50 × – 150 ×
Existing C/C++ legacy algorithm	Need direct integration without IPC	Cython / CFFI (<code>with nogil:</code>)	100 × – 200 ×
Multi-process memory bloat	<code>pickle</code> serialization over IPC sockets	<code>multiprocessing.shared_memory</code>	Eliminates 100% of IPC transfer lag
Threaded CPU tasks blocked by GIL	Pre-3.13 CPython GIL serialization	Python 3.13 Free-Threaded (<code>python3.13t</code>)	Near-linear multi-core scaling
Massive matrix multiplication / deep learning	CPU memory bandwidth exhaustion	PyTorch / Triton on NVIDIA CUDA GPU	1,000 × +

Exercises

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:



Attempt first; the reference solutions are in `exercises/_answers/ch18.py`.

Further Reading

- [NumPy documentation](#)
- [Numba documentation](#)
- [multiprocessing.shared_memory](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Why are NumPy loops over arrays vectorised?
- ▶ 2. What is `numba`'s `@njit` doing?
- ▶ 3. What does shared memory (`multiprocessing.shared\_memory`) avoid?
- ▶ 4. What limits GPU speedups for small workloads?